

A Extended Background and Mathematical Formulations

This section provides the detailed mathematical formulations and extended background for Battery-Free Sensor Nodes (BFNs), intermittent computing, and energy-aware primitives, which support the theoretical foundation of the EnTrust architecture discussed in Section II of the main manuscript.

A.1 Battery-Free Sensor Nodes and Intermittent Computing

Battery-Free Sensor Nodes (BFNs) operate by harvesting ambient energy from sources such as radio frequency (RF), solar radiation, vibration, or thermal gradients. The harvested power $P_h(t)$ is inherently time-varying and can be modeled as a stochastic process:

$$P_h(t) = \eta \cdot P_{src}(t), \quad (1)$$

where η denotes the energy conversion efficiency and $P_{src}(t)$ represents the ambient energy source intensity.

Energy is temporarily stored in capacitors or supercapacitors. The stored energy $E(t)$ in a capacitor C with voltage $V(t)$ is given by:

$$E(t) = \frac{1}{2}CV^2(t). \quad (2)$$

A node becomes operational only when $E(t)$ exceeds a minimum operational threshold E_{on} :

$$E(t) \geq E_{on}. \quad (3)$$

BFNs follow a **charge-store-compute** execution model. During the charging phase, energy accumulates until the voltage reaches a turn-on threshold V_{on} ; during computation, energy is consumed until it drops below a cutoff threshold V_{off} . The usable energy per cycle is thus:

$$E_{usable} = \frac{1}{2}C(V_{on}^2 - V_{off}^2). \quad (4)$$

Due to limited E_{usable} , tasks may not complete within a single cycle, leading to **intermittent computing**. If a task requires energy E_{task} :

$$E_{task} > E_{usable} \Rightarrow \text{task interruption occurs.} \quad (5)$$

To maintain forward progress, systems adopt checkpointing or idempotent execution models. The expected number of re-executions for a task can be approximated as:

$$N_{retry} \approx \left\lceil \frac{E_{task}}{E_{usable}} \right\rceil. \quad (6)$$

This intermittent behavior introduces challenges in preserving execution state, ensuring data consistency, and minimizing redundant computation.

A.2 Energy-Aware Task Scheduling

In BFNs, task scheduling must explicitly incorporate energy dynamics. Let $E_{res}(t)$ denote the residual energy and E_i the energy required for task i . A basic feasibility condition for task execution is:

$$E_{res}(t) \geq E_i + E_{safe}, \quad (7)$$

where E_{safe} is a safety margin to prevent brownout during execution.

The scheduler aims to maximize system utility under energy constraints. This can be formulated as an optimization problem:

$$\max \sum_i U_i x_i \quad (8)$$

subject to:

$$\sum_i E_i x_i \leq E_{budget}, \quad x_i \in \{0, 1\}, \quad (9)$$

where U_i represents the utility (e.g., priority or importance) of task i , and E_{budget} is the available energy within a scheduling window.

Dynamic Voltage and Frequency Scaling (DVFS) further enables fine-grained energy control. The dynamic power consumption is:

$$P_{dyn} = \alpha C_L V^2 f, \quad (10)$$

where α is the switching activity factor, C_L is the load capacitance, V is the supply voltage, and f is the clock frequency.

Execution time and energy consumption exhibit a trade-off:

$$E_{task} = P_{dyn} \cdot T_{exec}, \quad T_{exec} \propto \frac{1}{f}. \quad (11)$$

Thus, reducing V and f lowers instantaneous power but may increase total energy due to prolonged execution under leakage, necessitating careful scheduling decisions.

A.3 Energy-Aware Security Primitives

Energy constraints also significantly impact security design, requiring primitives that are both lightweight and interruption-resilient.

Physically Unclonable Functions (PUFs) generate device-specific responses based on manufacturing variations. A PUF can be modeled as:

$$R = f_{PUF}(C, \delta), \quad (12)$$

where C is the challenge input and δ captures physical randomness. Due to environmental noise, repeated evaluations may produce slightly different responses:

$$R' = R \oplus n, \quad (13)$$

where n denotes noise. Reliability is typically quantified using intra-Hamming distance:

$$HD_{intra} = \mathbb{E}[d_H(R, R')], \quad (14)$$

necessitating error correction mechanisms such as fuzzy extractors.

Atomic Lightweight Cryptographic Library decomposes cryptographic operations into fine-grained atomic units. Let a cryptographic task consist of k atomic operations:

$$\mathcal{C} = \{op_1, op_2, \dots, op_k\}. \quad (15)$$

Each atomic operation op_j has bounded energy consumption:

$$E(op_j) \leq E_{atomic}^{max}. \quad (16)$$

This ensures that:

$$E_{atomic}^{max} < E_{usable}, \quad (17)$$

allowing each operation to complete within a single energy cycle.

Security strength can be modeled as a function of executed rounds:

$$S = g(r), \quad (18)$$

where r is the number of completed rounds. Under energy constraints, the system dynamically selects r such that:

$$r = \max\{r' \mid E(r') \leq E_{res}(t)\}. \quad (19)$$

This leads to an adaptive security policy:

$$\text{maximize } S \quad \text{subject to } E_{crypto} \leq E_{res}(t), \quad (20)$$

balancing confidentiality and energy availability. Overall, the atomic and reconfigurable design enables BFNs to maintain secure operation under intermittent power, providing a practical trade-off between energy consumption, execution progress, and cryptographic strength.

B Comprehensive Energy Consumption Models

This section details the comprehensive energy consumption models for both communication and local processing, which serve as the basis for the energy-aware scheduling decisions in Section IV of the main manuscript.

In energy-harvesting or battery-free terminal devices, the available energy supply is not only highly constrained but also exhibits strong temporal intermittency. The total energy dissipation of a battery-free node can be classified into two major functional domains: communication-related energy E_{comm} and on-device computation energy E_{local} .

$$E_{total} = E_{comm} + E_{local}. \quad (21)$$

B.1 Communication Energy Consumption Modeling

The total communication energy is decomposed according to:

$$E_{\text{comm}} = E_{\text{tx}} + E_{\text{rx}} + E_{\text{ctrl}} + E_{\text{sec}},$$

where each term captures a distinct category of communication operations.

B.1.1 Transmission Energy Consumption

The energy for transmitting N packets of payload size L is given by

$$E_{\text{tx}} = N \times \left((P_c + \eta \cdot P_{\text{te}}) \times \frac{L}{R} \right), \quad (22)$$

where the transmission duration $\frac{L}{R}$ encapsulates the impact of physical-layer rate R on temporal energy cost. The circuit power P_c contains a static baseline component P_{base} and a modulation-related dynamic component P_{mod} :

$$P_c = P_{\text{base}} + P_{\text{mod}}, \quad P_{\text{mod}} \propto R^\beta, \quad \beta \approx 1. \quad (23)$$

To maintain a target SNR at the receiver, the required transmission power must compensate for propagation loss:

$$P_{\text{te}} = \frac{\text{SNR} \cdot N_0 \cdot d^\alpha}{\eta \cdot h}. \quad (24)$$

Power amplifier efficiency follows a saturation response:

$$\eta = \eta_{\text{max}} \cdot (1 - e^{-\gamma P_{\text{te}}}). \quad (25)$$

The payload size and data rate are computed as:

$$L = \left(\left\lceil \frac{\text{size}}{\text{MTU}} \right\rceil \cdot B + \text{size} \right) \cdot 8, \quad (26)$$

$$R = B \cdot \log_2 \left(1 + \frac{S}{N + N_{\text{interf}}} \right) \cdot C. \quad (27)$$

B.1.2 Reception Energy Consumption

The reception energy model is

$$E_{\text{rx}} = N \cdot \left(P_{\text{rx}} \cdot \frac{L}{R} \right) + P_{\text{idle}} \cdot T_{\text{idle}}. \quad (28)$$

Low-Power Listening (LPL) replaces continuous reception with periodic channel sampling:

$$E_{\text{rx}}^{\text{LPL}} = N \cdot \left(P_{\text{rx}} \cdot \frac{L}{R} \right) + (P_{\text{wake}} T_{\text{wake}} + P_{\text{sleep}} T_{\text{sleep}}) \frac{T_{\text{idle}}}{T_{\text{cycle}}}. \quad (29)$$

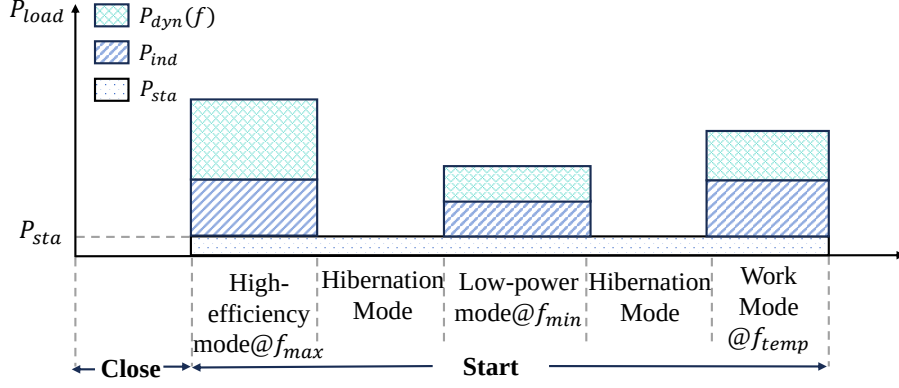


Figure 1: Overview of the LMAP authentication protocol for BFNs, showing the single broadcast-response interaction and lightweight security mechanisms.

B.1.3 Protocol Control Overhead

Control signaling includes handshake packets, channel probing, ACK/NACK exchange, and synchronization preambles:

$$E_{\text{ctrl}} = N_{\text{ctrl}} \left(\frac{P_{\text{tx}}C}{R} + \frac{P_{\text{rx}}C}{R} \right) + E_{\text{proc}}N. \quad (30)$$

In LPL-enabled nodes, additional wake-ups multiply this overhead:

$$E_{\text{ctrl}}^{\text{LPL}} = N_{\text{wake}} \left[N_{\text{ctrl}}^{\text{per}} \frac{(P_{\text{tx}} + P_{\text{rx}})C}{R} + E_{\text{proc}}N_{\text{ctrl}}^{\text{per}} \right]. \quad (31)$$

B.1.4 Secure Transmission Overhead

Security operations introduce substantial computational energy:

$$E_{\text{sec}} = N \cdot (E_{\text{enc}} \cdot L + E_{\text{dec}} \cdot L + E_{\text{auth}}). \quad (32)$$

If the system supports configurable security levels l , the generalized model becomes

$$E_{\text{sec}} = N \cdot (E_{\text{enc}}(l)L + E_{\text{dec}}(l)L + E_{\text{auth}}(l)). \quad (33)$$

B.2 Modeling of Local Processing Energy Consumption

Local energy consumption originates from processor switching activities, memory accesses, leakage currents, and operation of peripheral units.

B.2.1 System Load Power Consumption

The processor’s power consumption consists of three major components:

$$P_{\text{dyn,sys}}(f) = \alpha C_{\text{eq}} V_{\text{op}}^2 f, \quad (34)$$

$$P_{\text{ind}} = P_{\text{mem}} + P_{\text{IO}} + P_{\text{aux}}, \quad (35)$$

$$P_{\text{sta}} = V \cdot I_{\text{leak}}. \quad (36)$$

B.2.2 Task Execution Time

For parallelizable operations:

$$T = \max \left(\frac{C}{f_{\text{cpu}}}, \frac{D_{\text{tx}} + D_{\text{rx}}}{B}, \frac{S}{R_{\text{sto}}} \right), \quad (37)$$

while serial operations satisfy:

$$T = \frac{C}{f_{\text{cpu}}} + \frac{D_{\text{tx}} + D_{\text{rx}}}{B} + \frac{S}{R_{\text{sto}}}. \quad (38)$$

C Low-Level Hardware Implementation of the HEDA Architecture

This section provides the detailed hardware implementation of the Hardware-Embedded Dynamic Adjustment (HEDA) architecture mentioned in Section IV.B of the main manuscript. It details the internal logic of the DVFS Manager, Voltage Controller, and Frequency Controller.

C.1 DVFS Scheduling Controller Overview

For BFNs, DVFS-enabled components—such as processing cores and task schedulers—generate voltage–frequency requests based on real-time energy states and workload demands. The DVFS controller comprises a manager, a voltage controller, and a frequency controller, which jointly orchestrate voltage–frequency adjustments to adapt system operation to dynamically varying energy conditions.

Upon receiving a request, the Manager consults a software-configured VF lookup table to obtain the target voltage and frequency for the desired performance level. It compares the current and target voltages to decide whether scaling is required, then sends commands to the voltage and frequency controllers.

The Voltage Controller adjusts supply levels by programming an off-chip PMIC via I²C, while the Frequency Controller configures PLL clock sources to reach the target operating frequency. Together, these controls enable BFNs to autonomously adapt system operation under limited energy availability and dynamically varying workloads.

C.2 DVFS Manager — Unified Request Management Unit

In BFNs, the system core module (DVFS MCU) generates voltage-frequency scaling requests based on energy states and computational loads, specifying a target performance level. The DVFS Manager then consults a software-configured voltage-frequency lookup table (VF_LUT) to retrieve the corresponding parameters. It compares these with the current system state to determine if scaling is required, and if so, issues commands with target parameters to the voltage and frequency Controllers.

To ensure stable operation, a configurable delay mechanism enforces that frequency switching occurs only after the voltage adjustment has fully settled, thereby mitigating potential operational risks. The DVFS Manager integrates several key functions:

- **Performance Level Decoding:** Receives DVFS requests and target levels, looks up voltage and frequency from a table, and compares with the current voltage to decide increase, decrease, or hold.
- **Control State Machine:** Applies priority logic (voltage up before frequency up, frequency down before voltage down) for stable transitions, with delay states to await voltage or frequency stabilization after writing to the PMIC register.
- **Timing Control:** Uses the `delay_cycles` register and counters to manage delay intervals, where actual delay = `delay_cycles` × clock period T . State transitions are gated by completion signals for safe operation.
- **Fault Handling:** Detects anomalies such as voltage adjustment failures or operational timeouts. In the case of voltage mismatches, the module sets `voltage_fault` and logs diagnostic information; for timeout events, a `timeout_cycles` counter is employed, and the system reverts to the idle state if the threshold is exceeded.
- **Asynchronous Interaction:** Ensures reliable communication across multiple clock domains by employing double flip-flop synchronizers. It manages critical signals, including `voltage_ack`, `voltage_fault`, and `freq_ack`, thereby preventing timing violations, metastability issues, and signal glitches.

C.3 Voltage Controller — Voltage Regulation Module

The voltage controller performs discrete voltage adjustments for BFNs. It receives requests and target values from the DVFS controller, programs the off-chip PMIC via I²C, and is managed by a state machine (Figure 2, Table 1). To ensure reliability, it supports masked writes and read-back verification, providing accurate transmission and error feedback to upper layers.

An APB interface is provided, allowing the CPU to write start, stop, and acknowledge commands for I²C timing control into internal registers; address

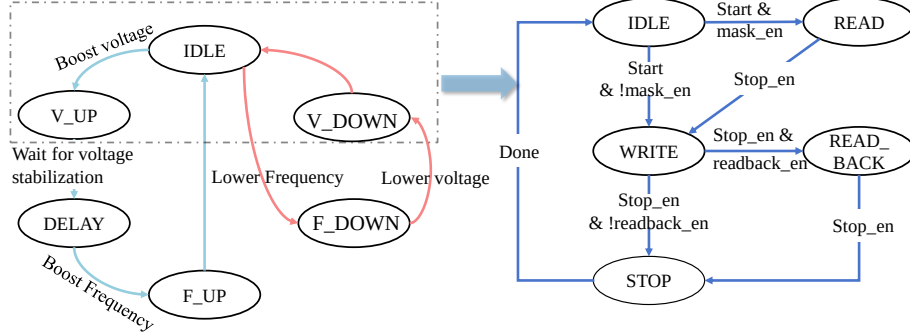


Figure 2: State machine of the voltage controller module, illustrating discrete voltage adjustment operations, communication with off-chip PMIC registers via I²C, and mechanisms for masked write and read-back verification.

Table 1: Descriptions of DVFS Control States

State Name	Function Description
IDLE	Idle state, waiting for a new DVFS adjustment request
VOLT_UP	Voltage increase phase, sending target voltage to the regulator module
DELAY_UP	Delay after voltage increase, waiting for voltage stabilization
FREQ_UP	Frequency increase phase, sending target frequency to the frequency module
FREQ_DOWN	Frequency decrease phase, reducing frequency as a priority
DELAY_DOWN	Delay after frequency decrease, waiting for frequency stabilization
VOLT_DOWN	Voltage decrease phase, lowering voltage accordingly

and data registers are used for PMIC register mapping and voltage data transmission.

Masked Write Mechanism: To prevent unintended modifications to unrelated fields in multi-bit PMIC registers, the module supports an optional masked write mechanism. This technique first reads the current register value (*mdata*) and then computes the actual value to be written (*wdata*) using the following equation:

$$wdata = (mdata \wedge mask_value) \vee (data0 \wedge \neg mask_value) \quad (39)$$

Finally, *wdata* is written back to the register. This method improves protocol compatibility and enhances portability.

Read-back Verification: After writing the target voltage value (*data0*)

Table 2: Voltage Controller Error Output Conditions

Functionality	Error Condition	Error Output
Basic write only	I ² C bus no response or write failure	1
Read-back verification enabled	$rdata \neq data0$	1
Masked write enabled	I ² C write phase failure	1
Masked write and Read-back verification	$rdata \neq wdata$	1
Successful operation	Bus responds correctly, data matches	0

to the PMIC register, the module reads back the register content (*rdata*) and compares it with *data0*. A match confirms that the write operation has completed correctly, whereas a mismatch asserts an error signal ($err = 1$) to report a write failure.

Status Feedback: The error signal *err* is synchronized with corresponding acknowledge signals and determined by active functional configuration. The specific conditions under which the error signal is generated are summarized in Table 2.

C.4 Frequency Controller — Clock Management Unit

The Frequency Controller enables dynamic performance adjustment for BFNs following energy restoration by supplying configurable clock signals to the DVFS MCU. It supports flexible clock source selection and single-stage frequency division, ensuring low-power, glitch-free transitions while maintaining overall system stability.

Upon receiving frequency configuration commands (**freq_cfg**) from the DVFS manager, the frequency controller selects one of five external PLLs via glitch-free multiplexing, disabling the current clock before enabling the target. A lightweight divider, controlled by **freq_cfg[15:8]**, supports 1/2, 1/4, and 1/8 ratios using three cascaded T flip-flops. All outputs pass through the glitch-free selector, ensuring stable and reliable clock management for BFNs.

D Energy-Aware Authentication Framework and Protocol Instantiation

To address the heterogeneity of IoT devices in terms of energy availability, computational capability, and security requirements [?, ?], this paper proposes an energy-aware hierarchical authentication framework.

The framework provides a unified abstraction that organizes heterogeneous authentication primitives, including lightweight hashing, multifactor authen-

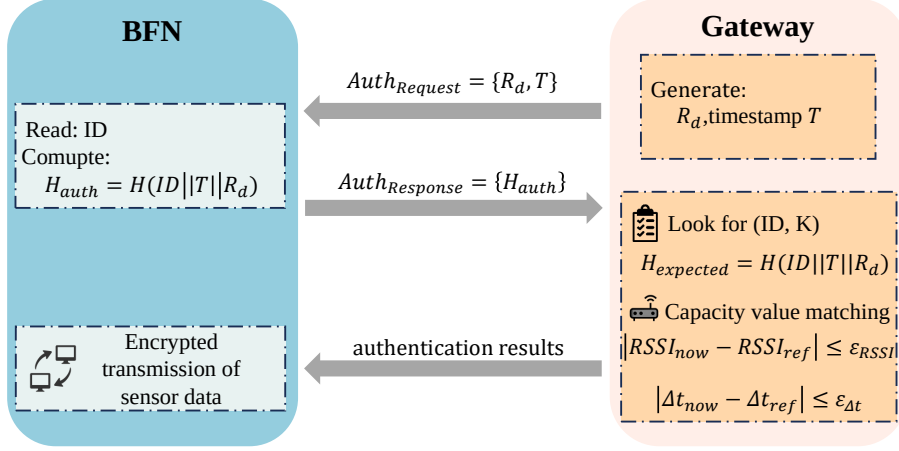


Figure 3: Overview of the LMAP authentication protocol for BFNs, showing the single broadcast-response interaction and lightweight security mechanisms.

tication, and PUF-based mechanisms, according to device energy states and hardware capabilities. This abstraction enables flexible and scalable identity authentication for devices ranging from passive sensors to moderately resource-constrained nodes under ultra-low-power conditions.

Based on this framework, three representative authentication protocols are developed for different operating regimes:

- **Lightweight Multi-factor Authentication Protocol (LMAP):** Designed for ultra-low-power and intermittently active nodes, this protocol leverages symmetric key primitives to reduce computational and communication overhead.
- **Light PUF-based Authentication Protocol (LPUFA):** This protocol employs PUF-based challenge-response mechanisms with lightweight error correction, making it suitable for hardware-constrained edge devices.
- **Enhanced Energy-aware Hash Authentication Protocol (E²HA):** Based on dynamic hash chains, this protocol adjusts authentication strength according to real-time energy availability.

Together, these protocols provide a hierarchical authentication solution that adapts to diverse energy and security conditions in low-power IoT environments.

D.1 Framework Definition and Design Principles

The proposed framework is characterized by three key aspects: energy-awareness, capability matching, and security scalability. Authentication complexity is adapted to instantaneous energy availability, protocol selection is aligned with device hardware features (e.g., PUF support and storage capacity), and authentication strength is scaled according to security requirements.

Formally, the framework can be expressed as a mapping:

$$\mathcal{P} = f(E, C, S) \quad (40)$$

where E denotes the energy state, C represents device capability, and S indicates the required security level.

D.2 Lightweight Multi-factor Authentication Protocol

The lightweight multi-factor authentication protocol (LMAP) is designed for energy-constrained and intermittently powered sensor nodes, particularly battery-free nodes (BFNs) relying on ambient energy harvesting. To minimize communication and computational overhead, the protocol adopts a single-challenge single-response mechanism, while integrating cryptographic verification with physical-layer feature binding to achieve robust and low-cost authentication. The overall workflow is illustrated in Fig. 4.

D.2.1 Gateway Broadcasts Authentication Request

Due to the intermittent operation of BFNs caused by unstable energy availability (e.g., RF, light, or thermal sources), frequent communication should be avoided. Therefore, the gateway periodically or on-demand broadcasts a lightweight authentication request to trigger node responses without requiring continuous listening. The request is defined as

$$\text{Auth_Request} = \{R_d, T\}, \quad (41)$$

where R_d is a session-specific random challenge generated by the gateway, and T is a timestamp indicating the freshness of the request. The timestamp is validated within a predefined time window to prevent replay attacks.

D.2.2 Terminal Computes Response and Sends Upstream

Upon receiving the authentication request, the terminal node immediately computes a lightweight authentication response using its locally stored identifier ID and secret key K . The authentication hash is generated as

$$H_{\text{auth}} = H(ID \parallel K \parallel T \parallel R_d), \quad (42)$$

where $H(\cdot)$ denotes a hardware-friendly lightweight hash function. The terminal then transmits the response frame

$$\text{Auth_Response} = \{ID, H_{\text{auth}}\} \quad (43)$$

to the gateway. This design ensures that the gateway can directly identify the responding node without exhaustive search, thereby reducing verification latency and computational cost.

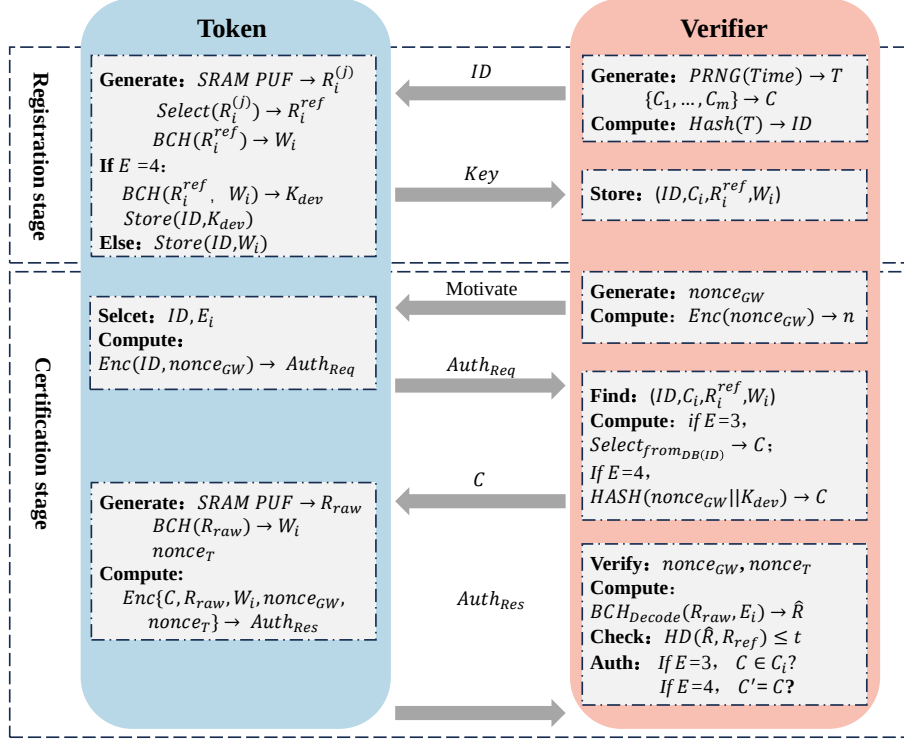


Figure 4: Protocol flow of the Lightweight PUF Authentication(LPUFA).

D.2.3 Gateway Performs Multi-factor Authentication Check

Upon receiving the response, the gateway first checks whether the timestamp T is within a valid time window to ensure message freshness. It then retrieves the corresponding credential pair (ID, K) from the whitelist and recomputes the expected hash value as

$$H_{expected} = H(ID \parallel K \parallel T \parallel R_d). \quad (44)$$

Authentication proceeds only if $H_{expected} = H_{auth}$, ensuring cryptographic correctness.

To further enhance security, physical-layer features are incorporated as an additional authentication factor. Specifically, the gateway measures the received signal strength (RSSI) and response delay Δt_{now} , and compares them with the reference values $RSSI_{ref}$ and Δt_{ref} , which are established through historical averaging over multiple successful authentications. The verification conditions are given by

$$\begin{aligned} |RSSI_{now} - RSSI_{ref}| &\leq \varepsilon_{RSSI} \\ |\Delta t_{now} - \Delta t_{ref}| &\leq \varepsilon_{\Delta t} \end{aligned} \quad (45)$$

where $\varepsilon_{\text{RSSI}}$ and $\varepsilon_{\Delta t}$ are tolerance thresholds determined based on statistical variations under normal operating conditions.

Authentication is considered successful only when both cryptographic verification and physical-layer constraints are simultaneously satisfied. Otherwise, the request is rejected, and the event can be logged for anomaly detection or subsequent re-authentication.

D.2.4 Security Analysis and Correctness

The correctness of LMAP follows from the deterministic consistency of the hash function, such that

$$H(ID \parallel K \parallel T \parallel R_d)_{\text{device}} = H(ID \parallel K \parallel T \parallel R_d)_{\text{gateway}}. \quad (46)$$

Security is ensured via joint cryptographic and physical design. Timestamp T guarantees replay resistance by enforcing freshness constraints. Secret key K ensures adversaries cannot forge valid authentication responses without legitimate credentials, providing strong impersonation resistance. Integrating physical-layer features like RSSI and response delay binds logical identities to physical transmission characteristics, offering extra protection against device cloning and relay attacks. Consequently, LMAP achieves dual-factor authentication that balances security robustness and energy efficiency, suiting resource-constrained, intermittently powered IoT environments.

D.3 Lightweight Physical Unclonable Function Authentication Protocol (LPUFA)

LPUFA exploits the inherent uniqueness and low-power characteristics of PUF [?] circuits to realize a physically attack-resistant authentication scheme suitable for moderately energy-constrained devices. The protocol operates in two phases, namely registration and authentication, during which end devices are not required to store cryptographic keys or perform conventional encryption and decryption operations. Instead, identity verification is accomplished by leveraging pre-collected challenge-response pairs (CRPs) in conjunction with lightweight error-correction mechanisms. This design reduces energy consumption and computational overhead while preserving strong hardware adaptability and security assurances. The complete protocol flow is illustrated in Figure 4.

D.3.1 Interactive Authentication Phase

When a passive Token accesses the network or initiates task execution, it performs secure authentication under temporary external power, following a standard Challenge-Response protocol outlined below:

- a) **Auth Request:** On power-up, the Token reads its device identifier from non-volatile memory, generates a random session nonce nonce_T , and transmits the request:

$$\text{Auth_Request} = \{\text{ID}, \text{nonce}_T\} \quad (47)$$

Table 3: Notation and Definitions in LPUFA Protocol

Symbol	Definition
Token	Passive device with embedded PUF; no long-term energy storage; operates only when externally powered.
Verifier	Authentication server with sufficient computation and storage resources.
ID	Unique identifier assigned to each Token by the Verifier.
C_i	The i -th challenge vector, randomly generated to stimulate the PUF.
$R_i^{(j)}$	Raw PUF response to C_i at the j -th sampling.
R_i^{ref}	Reference response for C_i , obtained via majority voting over multiple samplings.
C, R	Challenge and corresponding response of the PUF.
DB	Challenge-Response Pair (CRP) database storing $\langle ID, \{C_i, R_i\}_i \rangle$.
t	Error tolerance threshold to handle occasional jitter in PUF responses.
n	Number of challenges used per authentication session.

- b) **Challenge:** Upon receiving the request, the Verifier retrieves the ID's record from the database, generates its own session nonce nonce_{GW} , and selects a challenge C_i along with its helper data W_i .

- *Level 3 (Standard):* Randomly select an unused tuple $\langle C_i, W_i \rangle$ from stored CRPs.
- *Level 4 (Dynamic):* Dynamically generate a new challenge C_i and predict R_i^{ref} using a pre-trained Strong PUF machine learning model at the Verifier side.

The Verifier then sends the challenge message:

$$\text{Auth_Challenge} = \{C_i, W_i, \text{nonce}_{GW}\} \quad (48)$$

- c) **Response:** The Token applies the received challenge C_i to its PUF circuit to obtain the raw response R_{raw} . It then uses the received helper data W_i to correct noise-induced errors via lightweight BCH decoding, recovering the ideal response $R' = \text{BCH_Decode}(R_{raw}, W_i)$. To prevent exposing the PUF response in plaintext, the Token computes a cryptographic hash and sends the response:

$$V_T = \text{HASH}(R' \parallel \text{nonce}_{GW} \parallel \text{nonce}_T) \quad (49)$$

$$\text{Auth_Response} = \{V_T\} \quad (50)$$

- d) **Verification:** Upon receiving the response, the Verifier computes the expected hash value using its stored reference response R_i^{ref} :

$$V_V = \text{HASH}(R_i^{ref} \parallel \text{nonce}_{GW} \parallel \text{nonce}_T) \quad (51)$$

The Verifier checks if $V_T \stackrel{?}{=} V_V$. If they match, the Token is authenticated successfully.

This design decouples CRP collection from the runtime authentication process and adheres to the “compute offloading and storage delegation” philosophy. It provides robust security without storing cryptographic keys by leveraging un-cloneable physical characteristics to resist cloning, replay, and modeling attacks, making it particularly suitable for low-power, passive IoT deployments.

D.3.2 Authentication Correctness

Authentication is accepted if and only if the following condition holds: $V_T = V_V$. This single verification step elegantly ensures that:

- **Authenticity:** The response originates from the genuine PUF instance capable of reproducing R_{raw} close enough to R_i^{ref} .
- **Robustness:** Noise-induced errors in R_{raw} are tolerable and successfully corrected by W_i within the BCH threshold t .
- **Freshness:** The inclusion of both nonce_T and nonce_{GW} in the hash prevents replay attacks and challenge forgery from malicious adversaries.

D.4 Energy-Aware Efficient HMAC-Based Mutual Authentication Protocol (E²HA)

With the rapid proliferation of IoT devices, BFNs face stringent energy constraints, particularly in passive or semi-passive scenarios with unstable power and limited computing capability. Conventional authentication protocols impose heavy computation and long communication, leading to energy waste and frequent failures under fluctuating energy supply.

To address this, we propose E²HA, a lightweight HMAC-based authentication [?] protocol employing symmetric pre-shared keys for efficient identity binding and mutual authentication. The terminal energy level is dynamically incorporated to adapt authentication behavior and enforce security policies, enabling full authentication only when sufficient energy is available. The complete protocol flow is shown in Figure 5, making E²HA well suited for low-power IoT deployments(e.g., wireless sensor networks and edge computing).

D.4.1 Initialization

The terminal device (Device) and the gateway (Gateway) share a unique device identifier ID and an HMAC pre-shared key K_{psk} (a fixed secret key) during the initialization phase.

D.4.2 Energy-Aware Authentication Policy

To explicitly incorporate energy awareness into the authentication process, the device dynamically selects its authentication mode based on its current energy level E .

$$\text{Mode} = \begin{cases} \text{Full Authentication,} & E \geq E_{\text{high}} \\ \text{Lightweight Authentication,} & E_{\text{low}} \leq E < E_{\text{high}} \\ \text{Deferred Authentication,} & E < E_{\text{low}} \end{cases} \quad (52)$$

Full authentication executes the complete three-way HMAC-based handshake, lightweight authentication reduces communication overhead by omitting the final confirmation from the device, and deferred authentication postpones the process until sufficient energy is available. This design aligns protocol execution with instantaneous energy availability, minimizing unnecessary failures and energy depletion.

D.4.3 Authentication Process

If $E < E_{\text{low}}$, the device enters the Deferred mode and remains in sleep state. Otherwise, the protocol proceeds as follows:

- a) **Device Sends Authentication Request:** The device generates a random nonce R_d , reads the current timestamp T , and transmits the request in plaintext to allow gateway lookup, accompanied by an HMAC for integrity and authenticity:

$$\text{Auth_Request} = \{ID, T, R_d, \text{Auth_Code}, \text{Mode}\} \quad (53)$$

where $\text{Auth_Code} = \text{HMAC}(K_{\text{psk}}, ID \parallel T \parallel R_d \parallel \text{Mode})$.

- b) **Gateway Verifies Device Identity:** Upon receiving **Auth_Request**, the gateway checks if the timestamp T is within a valid time window to prevent replay attacks. It retrieves K_{psk} using the ID and computes:

$$\text{Auth_Code}' = \text{HMAC}(K_{\text{psk}}, ID \parallel T \parallel R_d \parallel \text{Mode}) \quad (54)$$

If $\text{Auth_Code}' \neq \text{Auth_Code}$, the connection is rejected.

- c) **Gateway Generates Challenge Response:** To prove its authenticity to the device, the gateway generates a random nonce R_g and computes

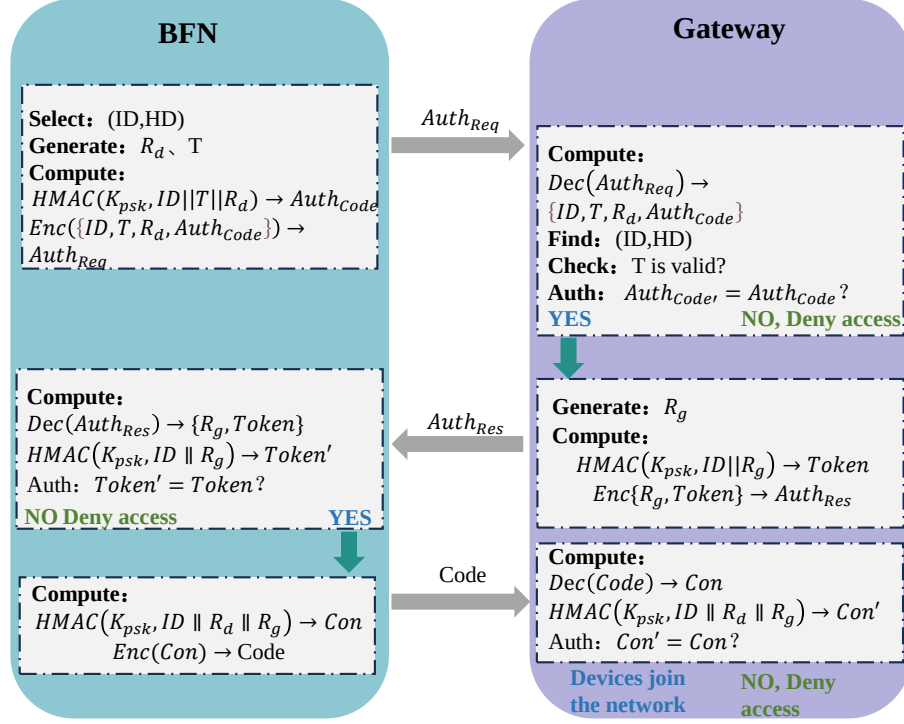


Figure 5: E²HA protocol flow, showing energy-aware mutual authentication between terminals and the gateway, with dynamic adaptation to terminal energy levels to reduce computation and communication overhead.

the HMAC token, binding it to the device's nonce R_d to ensure session freshness:

$$Token = HMAC(K_{psk}, ID || R_d || R_g || "S2C") \quad (55)$$

Here, the constant string "S2C" (Server-to-Client) acts as a directional tag for domain separation. It ensures that the gateway's token cannot be maliciously reflected back by an attacker as a valid device confirmation code. The gateway then sends $Auth_Response = \{R_g, Token\}$.

d) **Device Computes Token and Responds (Mode Dependent):** Upon receiving R_g , the device computes $Token'$ locally. If it matches $Token$, the gateway is authenticated.

- If Mode == Lightweight, the device stops here to save transmission energy. Mutual authentication is implicitly achieved.
- If Mode == Full, the device replies with a confirmation message to explicitly finalize the handshake:

$$Auth_Confirm = \{Confirm_Code\} \quad (56)$$

where $\text{Confirm_Code} = \text{HMAC}(K_{\text{psk}}, ID \parallel R_d \parallel R_g \parallel \text{"C2S"})$. The "C2S" (Client-to-Server) tag guarantees cryptographic separation from the gateway's token.

- e) **Gateway Verifies and Completes Authentication:** If the session is in Full mode, the gateway waits for Auth_Confirm and verifies:

$$\text{Confirm_Code}' = \text{HMAC}(K_{\text{psk}}, ID \parallel R_d \parallel R_g \parallel \text{"C2S"}) \quad (57)$$

If the codes match (or if the session was successfully negotiated as Lightweight), the device is granted network access.

D.4.4 Energy-Aware Verification Logic

Authentication success in E²HA is jointly determined by cryptographic validity and the declared energy mode, forming a dual-domain verification mechanism. Formally, the gateway accepts the authentication if and only if:

$$\begin{cases} \text{Auth_Code}' = \text{Auth_Code}, \\ \text{Confirm_Code}' = \text{Confirm_Code} & \text{if Mode} = \text{Full}, \\ E \geq E_{\text{low}}, \end{cases} \quad (58)$$

and the device accepts the authentication if and only if:

$$\text{Token}' = \text{Token}. \quad (59)$$

This conditional verification ensures both security and energy-aware feasibility. Security correctness is guaranteed by the use of timestamps T and device nonces R_d to prevent replay attacks, while directional tags ("S2C" and "C2S") prevent reflection attacks by ensuring that messages generated by the gateway and the device produce distinct HMAC values even with identical input parameters. Execution feasibility is achieved by dynamically pruning communication overhead: when $E_{\text{low}} \leq E < E_{\text{high}}$, the device omits transmission of Confirm_Code , reducing energy consumption while maintaining baseline mutual trust. Together, these mechanisms enable E²HA to adapt securely to real-time energy availability without compromising authentication integrity.

E Offline Registration Phase of the LPUFA Protocol

This section details the offline PUF registration phase that occurs before the deployment of the battery-free nodes, which serves as the prerequisite for the interactive authentication phase described in Section V.C of the main manuscript.

Before deployment, each terminal device must complete PUF modeling and identity binding within a secure offline environment. This phase is carried out collaboratively by the Verifier and the new Token, as described below:

- a) **ID Assignment and Initialization:** The Verifier generates a unique ID for each Token and writes it into the Token’s read-only memory.
- b) **CRP Collection:** The Verifier generates a pseudo-random set of challenges $\{C_1, \dots, C_m\}$ and applies them to the PUF circuit in the Token. Considering the potential impact of environmental factors such as voltage fluctuations and temperature variations, a *Jitter Measurement* mechanism is introduced to accurately characterize the stability of the PUF responses:
 - For each challenge C_i , the Verifier powers or resets the Token multiple times and collects the response $R_i^{(j)}$ for $j = 1, 2, \dots, K$.
 - Each bit’s stability is computed, and a reference response R_i^{ref} is formed via majority voting.
 - Unstable bits are explicitly labeled as low-confidence indicators to facilitate efficient error correction during authentication procedures.
- c) **Helper Code Generation:** Considering bit errors in practical deployments, each R_i^{ref} is encoded using BCH codes to generate helper data W_i , with decoding threshold t negotiated during system initialization.
- d) **Database Storage:** The Verifier securely stores the tuple $\langle ID, C_i, R_i^{ref}, W_i \rangle$ in the database. The Token does not store any CRPs or helper data locally, thereby minimizing storage overhead and preventing physical extraction attacks.

After registration, the Token is deployed in the field. It does not store any cryptographic keys or CRPs, retaining only its device identifier and minimal lightweight hardware state. When external energy becomes available (e.g., via RF power), the Token is briefly powered on to compute real-time PUF responses, thereby minimizing both energy consumption and information leakage and making it particularly suitable for passive sensor node applications.

F Theoretical Security Analysis of Authentication Protocols

Based on the reviewers’ suggestions, we provide a comprehensive theoretical security comparison of the three instantiated lightweight authentication protocols (LMAP, LPUFA, and E2HA) discussed in Section V of the main manuscript. This analysis evaluates their resistance to common attack vectors in energy-constrained environments.

To provide a clear overview of the security properties of three representative lightweight authentication protocols for highly energy-constrained passive terminal devices, we present a qualitative comparison based on each protocol’s interaction logic and underlying mechanisms. Table 4 summarizes the resistance of each protocol to common attack vectors, including replay attacks, man-in-the-middle attacks, impersonation, key leakage, physical attacks, and energy-related

Table 4: Security capability comparison of lightweight authentication protocols

Attack Type	LMAP	LPUFA	E²HA
Replay Attack	✓	✓	✓
Man-in-the-Middle	×	✓	✓
Impersonation Attack	✓	✓	✓
Key Leakage Attack	✓	✓	×
Replay and Delay Attack	✓	✓	✓
Physical Attack	×	✓	✓
Energy-Constrained Attack	✓	✓	✓
Redundant Authentication	✓	✓	✓
Eavesdropping	×	✓	✓
Tampering Attack	×	×	✓
Mutual Authentication	×	×	✓

threats. Note that this comparison is based on protocol design and mechanism analysis rather than empirical testing.

The qualitative analysis considers key mechanisms such as timestamps, nonces, PUF responses, HMAC codes, and physical layer detection, as well as energy management strategies. LMAP provides basic protection against replay and key attacks; LPUFA adds resistance to man-in-the-middle, physical, and eavesdropping threats through PUF-based authentication; and E²HA offers full mutual authentication along with robust defense against tampering and MITM. This comparison highlights the trade-offs among complexity, security coverage, and energy efficiency in ultra-low-power IoT deployments.

Summary

This appendix provides supplementary details of the proposed energy-driven task scheduling mechanism, including implementation examples, data analysis, and experimental results. The additional figures and explanations are intended to complement the main body of the paper by clarifying design choices, demonstrating performance trade-offs, and validating the effectiveness of the approach under dynamic energy conditions. Together, these results further confirm the practicality and adaptability of the proposed mechanism in energy-constrained IoT environments.